

Q1 What is a Context Model?

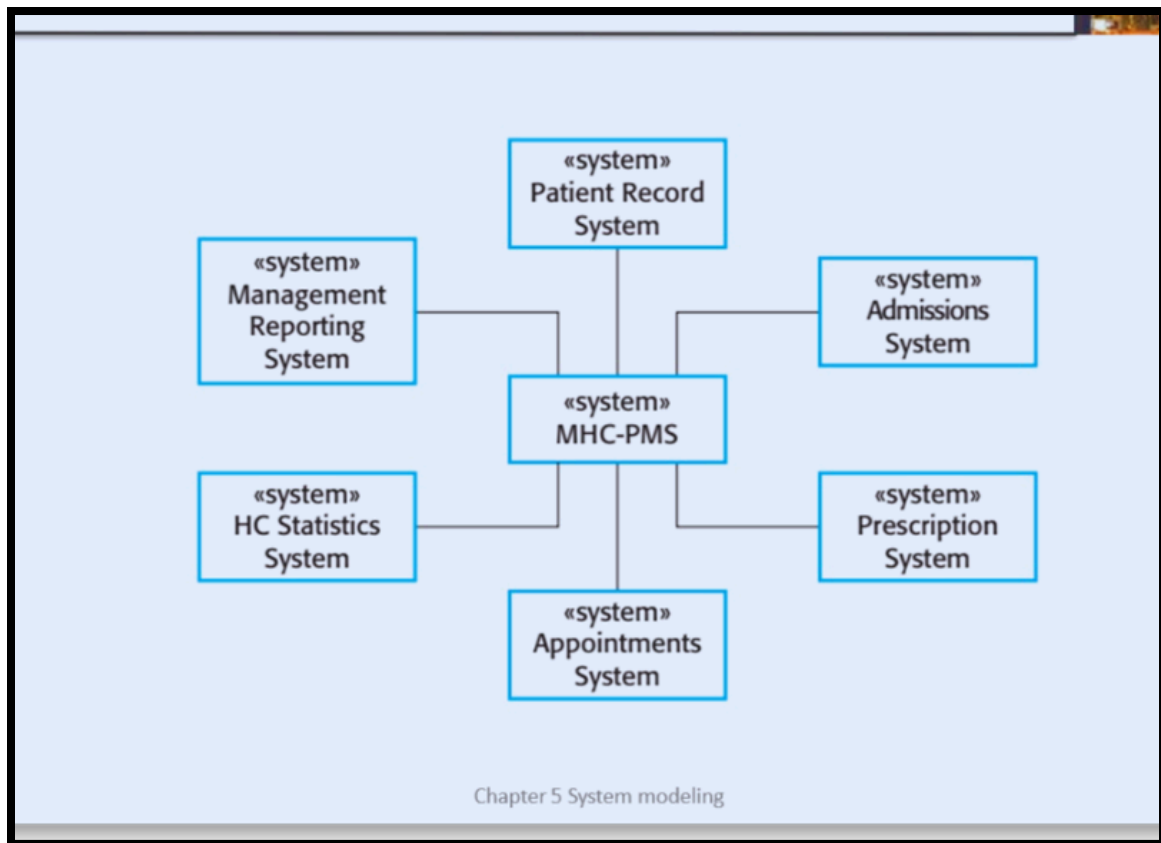
A context model is a high-level architectural view that shows the system as a single entity surrounded by other systems and external entities with which it interacts. It does NOT show the internal structure of the system – only its relationships with the external environment.

The Context Model for MHC-PMS

MHC-PMS (Mental Health Care Patient Management System) is a specialized information system used in clinics that treat patients with mental health problems

External Systems

1. **Admissions System**
 - Manages patient admission and bed allocation
 - MHC-PMS queries it when admission is required
2. **Prescription System**
 - Handles medicine prescriptions
 - MHC-PMS sends prescription requests
3. **Patient Record System**
 - Stores complete patient health records
 - MHC-PMS retrieves and updates data
4. **Statistics System**
 - Generates reports and analysis
 - Used by management and health authorities
5. **Appointments System**
 - Manages scheduling of patient visits
 - MHC-PMS coordinates appointments
6. **Staff Management System**
 - Stores staff details and availability
 - Helps assign doctors and staff



A A P P S M

- **A** → Appointments
- **A** → Admissions
- **P** → Patient Record
- **P** → Prescription
- **S** → Statistics
- **M** → Management Reporting

Advantages of Context Models

1. Provides a clear overview of system interactions at a glance.
2. Helps stakeholders understand what data flows in and out.
3. Useful for identifying integration requirements early.
4. Serves as a communication tool between technical and non-technical stakeholders

Sequence Diagram – View Patient Information

Definition

A **sequence diagram** shows how different parts of a system interact with each other step by step over time. It shows the order in which messages are passed.

Components of Sequence Diagram

1. **Objects / Participants**

These are the people or system parts involved (like user, UI, database).

2. **Lifeline**

A vertical dashed line that shows how long an object exists during interaction.

3. **Activation Bar**

A small rectangle that shows when an object is working or processing.

4. **Messages**

Arrows that show communication between objects.

5. **Combined Fragments**

Used to show conditions like if-else or loops.

O
L
A
M
C

Participants in this diagram

- Medical Receptionist
- User Interface (UI)
- Controller
- Authorization System
- Database

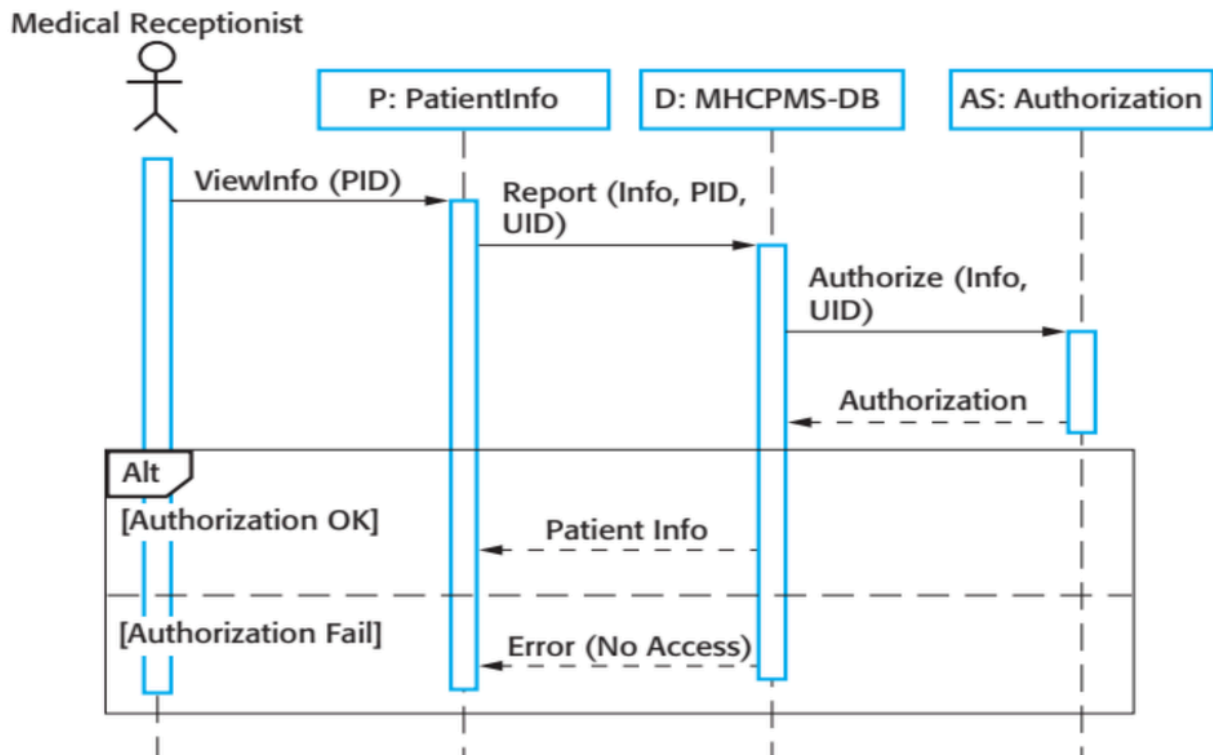
Flow of Interaction

1. Receptionist enters patient ID
2. UI sends request to Controller
3. Controller checks user authorization
4. Authorization system verifies user
5. If valid, Controller requests data from Database
6. Database sends patient data
7. Controller sends data to UI

8. UI displays information to Receptionist

Diagram

Sequence diagram for View patient information



modeling

Explanation

The receptionist enters the patient ID through the UI. The request is sent to the controller, which checks authorization. If the user is valid, the system retrieves data from the database and displays it.

Conclusion

Thus, the sequence diagram shows how system components interact step by step to provide patient information securely.

Data Transfer Use Case- Overview

This use case involves a Medical Receptionist transferring patient data from the local MHC-PMS database to a national patient database (Patient Record System).

Purpose: To update central national health records with new or updated patient information from the local mental health clinic.

Actors Involved

- **Primary Actor- Medical Receptionist:** The human user who initiates and controls the data transfer process.
- **Secondary Actor- Patient Record System (PRS):** The external national database system that receives the transferred data.

(Transfer Data Use Case Flow)

Here is the **Transfer Data** process in very simple words:

- Receptionist logs in to the MHC-PMS system using username and password.
- Receptionist chooses the patient whose data needs to be sent.
- System shows the patient’s information on screen so receptionist can check it.
- Receptionist clicks the “Transfer Data” button.
- System packs the patient data safely (makes it secure).
- System connects to the big hospital’s Patient Record System (PRS).
- Data is sent from MHC-PMS to the external system.
- Patient Record System checks the data and accepts it.
- System shows a “Success” message to the receptionist.
- System records everything in a log (for safety and checking later).

Tabular Description of “Transfer Data” Use Case	
Field	Description
Use Case	Transfer Data
Actors	Medical Receptionist, Patient Record System (PRS)
Description	A receptionist transfers data from MHC-PMS to a central patient record database maintained by a health authority. The data may include updated personal details (address, phone, etc.) or a summary of diagnosis and treatment.
Data	Patient’s personal information, treatment summary
Stimulus	Command given by the medical receptionist
Response	Confirmation that PRS has been updated
Comments	Receptionist must have proper security permissions to access and transfer data

Diagram



High-Level Architecture (Easy + Full Marks)

High-level architecture means the overall structure of a system and its main parts (subsystems) and how they work together.

The weather station is a remote automatic system that collects weather data like temperature, wind, and rain, and sends it to a central system. Because it works in harsh environments without human help, its architecture is designed to be simple, reliable, and modular.

Design Considerations

The architecture is designed considering:

- Remote Operation → No human control
- Reliability → Works continuously (24/7)
- Power Efficiency → Uses limited battery/solar power
- Fault Tolerance → Continues even if parts fail
- Maintainability → Supports remote updates

Architecture Style

- System is divided into independent subsystems
- Each subsystem performs a specific function
- Subsystems communicate through interfaces/communication link
- Failure of one subsystem does not stop the whole system

Six Major Subsystems

1. Data Collection Subsystem

- Collects data from sensors
- Processes and formats data
- Buffers data before sending

2. Instruments Subsystem

- Connects to sensors
- Reads weather data (temperature, wind, rain)
- Handles communication with hardware

3. Communications Subsystem

- Sends data to external systems

- Receives commands
- Manages transmission errors

4. Fault Management Subsystem

- Detects system failures
- Recovers from errors
- Monitors system health

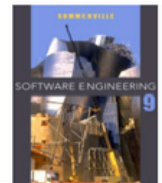
5. Power Management Subsystem

- Controls battery and power usage
- Manages energy saving
- Shuts down non-essential parts

6. Configuration Subsystem

- Stores system settings
- Handles updates and reconfiguration
- Maintains calibration data

Connector



High-level architecture of the weather station

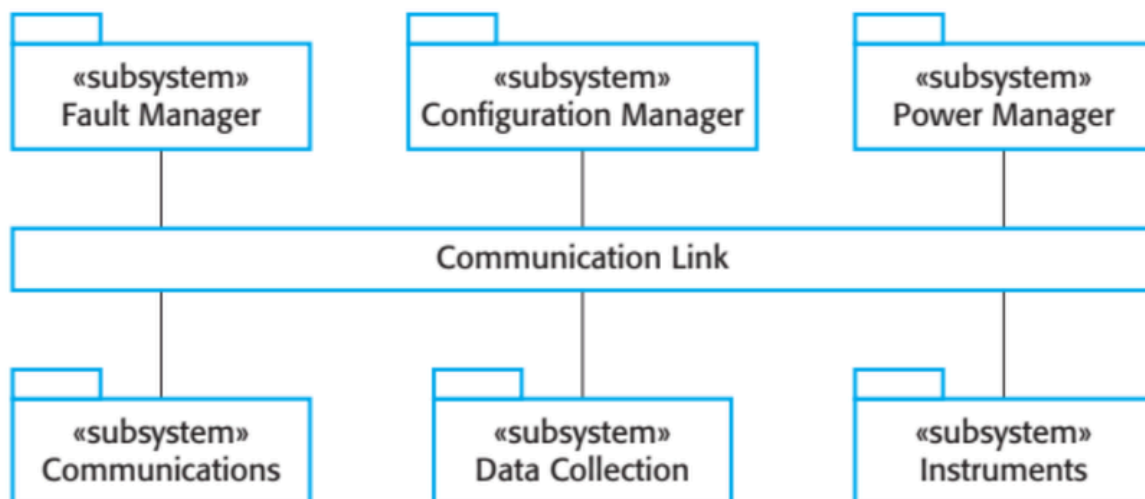


Figure 7.4 High-level architecture of the weather station

What is Agile Software?

Agile software development is a method of building software in **small parts (iterations)** with **continuous feedback from customers**.

It focuses on:

- Working software
- Fast delivery
- Adapting to changes

From your notes: Agile delivers software quickly and evolves with changing requirements

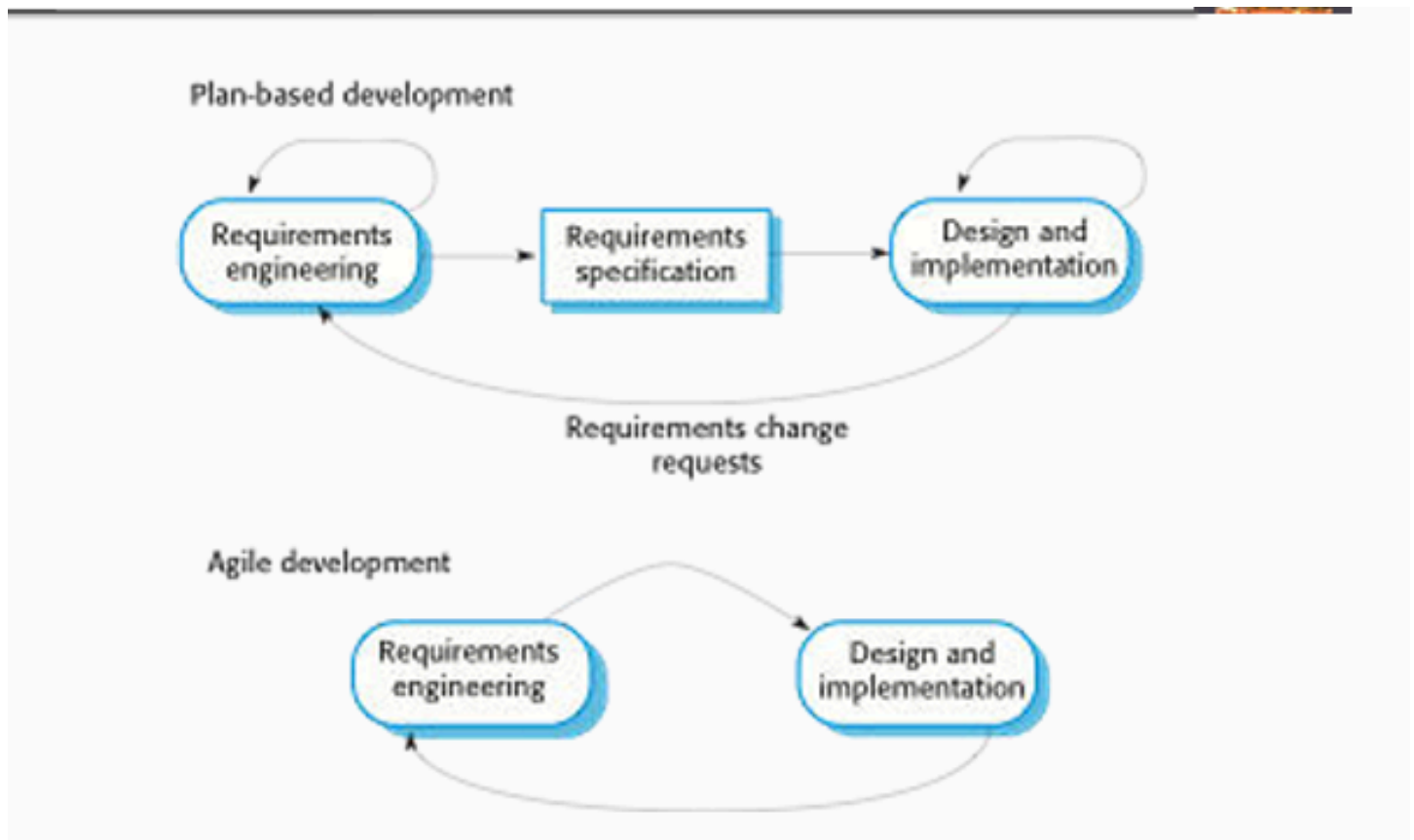
Why do we need Agile?

1. **Requirements keep changing**
→ Agile allows easy updates
2. **Fast delivery needed**
→ Software released in small parts
3. **Customer involvement**
→ Feedback improves product
4. **Reduces risk**
→ Errors found early
5. **Flexible development**
→ No strict fixed plan

Difference: Plan-Driven vs Agile

Feature	Plan-Driven	Agile
Approach	Fixed plan	Flexible
Development	Step-by-step	Iterative
Changes	Difficult	Easy
Customer Role	Less involvement	Continuous involvement
Documentation	Heavy	Minimal
Delivery	At end	Frequent
Suitable for	Stable requirements	Changing requirements

if u want u can do diagram



Key Point (write this line)

Plan-driven focuses on **planning first**,

Agile focuses on **adapting and delivering continuously**

Principles of Agile Methods



The principles of agile methods

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

OR

Principles of Agile Methods (Easy + Expanded)

1. Customer Involvementx

Customer is involved throughout development and gives regular feedback.
They help decide requirements and check each version of the system.
This ensures the final product meets user needs.

2. Incremental Delivery

Software is developed in small parts (increments) and delivered step by step.

Each increment adds new features.

This allows early delivery and quick feedback.

3. People not Process

Focus is on team skills and communication rather than strict rules.

Developers decide how to work efficiently.

Good teamwork improves productivity and quality.

4. Embrace Change

Changes in requirements are accepted even at later stages.

System is designed to handle changes easily.

This makes the software flexible and adaptable.

5. Maintain Simplicity

Keep the system and development process simple.

Avoid unnecessary features and complexity.

Simple design makes the system easier to develop and maintain.

Test-Driven Development (TDD)

Introduction

According to Software Engineering 9th Edition, **Test-Driven Development (TDD)** is a development approach in which **testing and coding are performed together**.

In this method, **tests are written before the actual program code**, and the code is developed to pass those tests. This approach is widely used in agile methods like Extreme Programming.

Core Principle

Write the test first, then write the code to pass the test

This ensures that every part of the program is tested from the beginning.

TDD Cycle (Red–Green–Refactor)

1. RED (Write Test)

A test is written for a small functionality. Since the code is not yet implemented, the test fails. This helps define what the program should do.

2. GREEN (Write Code)

The minimum code required is written to make the test pass. The goal is not to add extra features but just enough to satisfy the test.

3. REFACTOR (Improve)

The code is improved by removing duplication, simplifying logic, and improving readability. All tests are run again to ensure no errors are introduced.

This cycle is repeated for every new functionality.

TDD Process (Short Steps)

1. Identify a small functionality
2. Write a test for it
3. Run the test (it fails)
4. Write code to pass the test
5. Run test again (it passes)
6. Repeat for next feature

Benefits of TDD

1. Better Code Coverage

Every part of the program is tested because code is written only to satisfy test cases.

2. Regression Testing

All previous tests are stored and run again when changes are made. This helps detect if new code breaks existing functionality.

3. Easy Debugging

When a test fails, the error is usually in the small piece of code just written, making debugging easier and faster.

4. Documentation

Test cases act as documentation because they show how the code is expected to work and how it should be used.

5. Improved Design

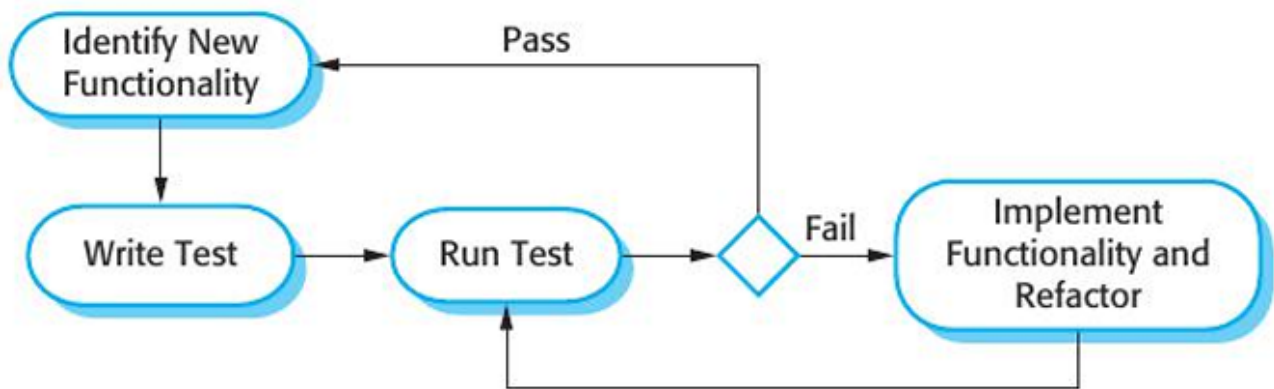
Writing tests first encourages modular and simple code. It improves code structure and separation of concerns.

Challenges of TDD

- Requires discipline and practice
- Writing tests takes extra time initially
- Not suitable for all systems (e.g., UI-heavy systems)

TDD vs Traditional Testing

TDD	Traditional
Tests written before code	Tests written after code
Testing is continuous	Testing is a separate phase
High test coverage	May have incomplete coverage
Bugs found early	Bugs found later



Classes and Association in MHC-PMS

Introduction

According to Software Engineering 9th Edition, **structural models** show the organization of a system in terms of its components and their relationships.

A **class diagram** represents the static structure of the system by showing classes and associations between them.

UML Classes in MHC-PMS

A **class** represents a real-world entity in the system and contains:

- Attributes (data)
- Methods (operations)

Each class is shown as a rectangle with 3 parts:

- Class name
- Attributes
- Methods

Key Classes in MHC-PMS

1. Patient

- Attributes: patientID, name, address, DOB
- Represents a person receiving treatment

2. Consultation

- Attributes: date, time, notes
- Represents a patient visit

3. General Practitioner (GP)

- Attributes: name, address
- Patient's family doctor

4. Condition

- Attributes: conditionName, severity
- Represents illness (e.g., depression)

5. Medication

- Attributes: drugName, dosage

- Represents prescribed medicines

Associations in MHC-PMS

An **association** shows the relationship between classes.
It is represented by a line connecting two classes.

Multiplicity (IMPORTANT)

Shows how many objects are related:

- **1** → exactly one
- **1..*** → one or more
- **0..*** → zero or more

Key Associations

1. Patient – Consultation (1..*)

- One patient can have many consultations
- Each consultation belongs to one patient

2. Patient – GP (1:1 / many-to-one)

- Each patient has one GP
- One GP can have many patients

3. Patient – Condition (1..*)

- One patient can have many conditions

4. Consultation – Medication (0..*)

- One consultation may have many medications

